

College Application Evaluator — Checkoff Dry Run

A test run of the TA checkoff process described in `labs/college_application_evaluator_manual.md`, performed against the reference solution `labs/solutions/college_application_reviewer.py`, to confirm the manual’s example output, self-check steps, and checkoff questions actually hold up against the real program before handing the lab to students.

Run 1 — matches the manual’s documented example exactly

Inputs: Taylor Morgan, Alabama, GPA 3.95, ACT 32, Math ACT 30, English ACT 27, 120 service hours, 4 activities, first-gen “no”, Engineering.

Output: identical, line for line, to the Example Output block in the manual (GPA/ACT MET, ADMITTED, automatic admission YES, \$12000 scholarship, Honors College ELIGIBLE, calculus track, advanced composition, leadership ELIGIBLE, first-gen NOT ELIGIBLE, OUT-OF-STATE, tuition waiver ELIGIBLE, Presidential Scholarship INVITED).

Verdict: PASS — the manual’s example output is accurate.

Run 2 — failing applicant, in-state, undeclared major

Inputs: Jamie Park, Mississippi, GPA 2.1, ACT 15, Math ACT 14, English ACT 16, 10 service hours, 1 activity, first-gen “yes”, Undecided.

Output: GPA/ACT NOT MET, NOT ADMITTED, automatic admission NO, \$0 scholarship, Honors College NOT ELIGIBLE, English placement DEVELOPMENTAL SUPPORT RECOMMENDED, leadership NOT ELIGIBLE, first-gen ELIGIBLE, IN-STATE — with the Engineering placement line, the out-of-state tuition waiver line, and the Presidential Scholarship line **all correctly absent**.

Verdict: PASS — this is the exact case the manual’s self-check asks students to try (a failing GPA/ACT applicant), and it confirms the three conditionally-silent sections (Milestone 5’s Engineering placement, Milestone 6’s tuition waiver, and Milestone 6’s Presidential Scholarship interview) correctly print nothing when their outer `if` is false, rather than crashing or printing an empty label.

Run 3 — scholarship-tier boundary check

Inputs: Riley Chen, Mississippi, GPA 3.7, ACT 28, Math ACT 25, English ACT 22, 60 service hours, 2 activities, first-gen “no”, Biology.

Output: Academic scholarship correctly lands on **\$8000** (the 3.7/28 tier), not \$12000 — confirming the cascading `if scholarship == 0`: guards correctly stop at the first tier a GPA/ACT pair actually qualifies for, rather than re-checking or double-awarding a lower tier. Honors College is ELIGIBLE at the same 3.7/28 boundary, matching Milestone 5’s stated threshold.

Verdict: PASS — this is the boundary case most likely to expose an off-by-one or missing-guard bug in a student’s scholarship cascade, and the reference solution handles it correctly.

Checkoff questions asked, and the answers a TA should expect

These are the kind of questions the manual’s Deliverable #4 says a TA will ask. Each is answered here against the reference solution, so a TA has a known-correct baseline before doing checkoffs with students.

1. **“Why is a nested if used for admission status instead of and?”** Because Milestone 3 explicitly requires nested `if` statements in place of `and/or` — the outer `if gpa >= 2.5:` contains an inner `if act >= 18:`, with a `NOT ADMITTED else` at both levels so every path prints exactly one admission line.
2. **“Why does the scholarship section start `scholarship = 0` and check if `scholarship == 0`: before each lower tier?”** Because the tiers must be evaluated highest to lowest without falling through to a lower tier once a higher one has already matched. Guarding each subsequent block on `scholarship == 0` is what prevents, e.g., a 3.95 GPA/32 ACT applicant from also being overwritten down to a lower tier by a later check.
3. **“Why does the Engineering placement section have no `else` at the outer level?”** Because Milestone 5 requires it to print *nothing* for a non-Engineering major, not a “not applicable” message — the outer `if major.lower() == "engineering":` has no matching `else`, only the inner `MET/NOT MET` branches have one.
4. **“Why is `.lower()` used on `major`, `first_generation`, and `state` before comparing?”** So that capitalization in the applicant’s typed answer (`"Engineering"`, `"ENGINEERING"`, `"engineering"`) doesn’t change the outcome — string equality in Python is case-sensitive, so comparing the lowered value is what makes the check robust to how a real applicant would type it.
5. **“Walk through why Riley Chen’s scholarship is \$8000 and not \$12000, given a 3.7 GPA and a 28 ACT.”** The \$12000 tier requires `GPA >= 3.9`, which 3.7 does not satisfy, so that block is skipped entirely; the code then checks `if scholarship == 0`: (true, since nothing has matched yet) and evaluates the \$8000 tier, where both `3.7 >= 3.7` and `28 >= 28` hold, so `scholarship` becomes 8000 and no later tier is checked because the guard now fails.

Overall result

All three runs matched expected behavior, the manual’s example output is verified accurate, and the reference solution can correctly answer every checkoff-style question above. The checkoff process described in `labs/college_application_evaluator_manual.md` is confirmed workable end-to-end.